

TL;DR: базовые структуры определяют цену чтения и записи

Table of contents

TL;DR: базовые структуры определяют цену чтения и записи	1
Зачем это прикладному инженеру	1
Типовые возражения	1
Итог	2

TL;DR: базовые структуры определяют цену чтения и записи

За модуль мы разобрали:

- **В-дерево** — движок, оптимизированный под предсказуемое чтение: сбалансированное дерево на диске, точечный и диапазонный поиск за $O(\log n)$.
- **LSM-дерево** — движок, оптимизированный под запись: append в memtable, sealed SSTable, фоновая compaction.
- **Bloom filter** — вероятностный фильтр, который за байты памяти отсекает заведомо отсутствующие ключи.
- **Хранение строк и индексы** — где лежат полные строки таблицы, чем heap отличается от clustered и почему каждый индекс живёт как отдельная структура.
- **Классические индексы** — чем отличаются B-tree, hash и bitmap, и почему индекс ускоряет одни запросы, но удорожает изменения данных.

Зачем это прикладному инженеру

Полезно ли до мельчайших деталей понимать, как при вставке перебалансируется В-дерево? Сомнительно.

Но когда читаешь документацию по БД или выбираешь между PostgreSQL и MongoDB, важно видеть, на какие компромиссы система идет под капотом, и не удивляться, когда «всё внезапно замедлилось».

А знание того, что каждый дополнительный индекс — это отдельное дерево, которое нужно обновлять при каждом INSERT, и вовсе бесценно.

Типовые возражения

«Массовая загрузка тормозит, давайте добавим ещё индексов» ведёт ровно не туда: в PostgreSQL каждый вторичный B-tree обновляется на каждом INSERT, и поэтому массовая загрузка с индексами почти всегда дороже. При массовой загрузке вторичные индексы обычно создают после загрузки; если таблица не обслуживает живой трафик,

обычный `CREATE INDEX` обычно быстрее, а `CREATE INDEX CONCURRENTLY` нужен именно для неблокирующего сценария.

Итог

Знание устройства — не для того, чтобы написать свою БД. Оно для того, чтобы понимать цену каждого `CREATE TABLE`, каждого `CREATE INDEX` и самого выбора БД под задачу.